

MAC-SQL Pro: Reinforced MAC-SQL Framework

Jiayu Wang, Joseph Stanley, Yung-Ching Yang, Zafeerah Sameen Subhani

University of Texas at Austin
Austin, USA

jiayuwang@utexas.edu
stanley.p.joseph@gmail.com
yungching68@utexas.edu
zafeerah.subhani@utexas.edu

Abstract

Recent progress in Large Language Models (LLMs) has improved Text-to-SQL generation. However, current open-source solutions often struggle with extensive schemas, multi-step reasoning, and limited interpretability. In this work, we present MAC-SQL Pro, an enhanced framework built upon the multi-agent MAC-SQL architecture. We address critical limitations in the baseline regarding schema hallucination, opaque reasoning, and execution errors. Our approach introduces (1) strict schema scoping with canonical matching to enforce faithful schema pruning, (2) a fine-tuned SQL-Llama model trained on a targeted synthetic dataset to mitigate aggregation and date-handling errors, and (3) an interactive UI that surfaces agent traces and supports real-time debugging. We evaluate on the Spider and BIRD benchmarks using Execution Accuracy (EX) and Valid Efficiency Score (VES) which emphasize execution-level correctness over exact string match. On our evaluation subsets, MAC-SQL Pro achieves 92.25% EX on Spider ($n = 200$) and 63.39% EX / 68.39% VES on BIRD ($n = 195$). The evaluation results show a practical path toward more robust and transparent Text-to-SQL systems. Code can be accessed at <https://github.com/JosephStan/MAC-SQL/tree/Jiayu>.

CCS Concepts

• **Computing methodologies** → **Natural language processing:** *Machine learning approaches.*

Keywords

Text-to-SQL, Multi-Agent Systems, Large Language Models, Interactive Systems, Database Management

ACM Reference Format:

Jiayu Wang, Joseph Stanley, Yung-Ching Yang, Zafeerah Sameen Subhani. 2026. MAC-SQL Pro: Reinforced MAC-SQL Framework. In . ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

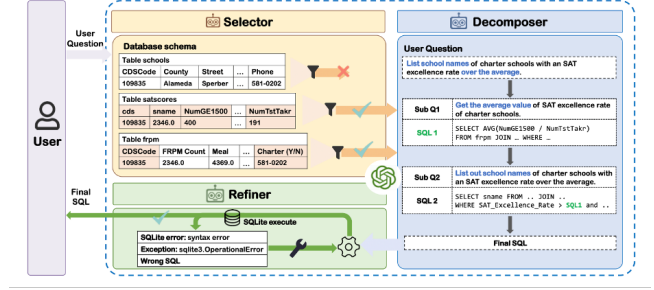


Figure 1: Overview of MAC-SQL framework

1 Introduction

Current open-source LLMs for Text-to-SQL continue to encounter difficulties when handling extensive databases, intricate user queries, and erroneous SQL results. First, the restricted context window of these models limits their ability to process large schemas. Second, single-pass SQL generation often fails on complex, multi-step queries. Third, the generated SQL lacks systematic verification, and refinements rely solely on execution errors rather than deeper reasoning.

Our baseline project adopts a multi-agent Text-to-SQL framework [4], which consists of three agents: a Selector to prune the schema, a Decomposer to break down complex queries, and a Refiner to validate and repair SQL. While this architecture is conceptually promising, several issues arose during implementation. First, the Selector consistently returned the entire schema rather than identifying relevant tables and columns, so the purpose of schema pruning was failed. Second, the system produced no meaningful intermediate outputs from the Decomposer or Refiner, which makes agent reasoning opaque and hinders debugging. Third, the framework lacks an operational interface, thus preventing effective inspection or iteration.

These limitations highlight a gap between the system's theoretical design and its practical usability. To address them, we introduced improved prompt engineering for the Selector and Decomposer, developed an interactive UI to visualize agent reasoning, and fine-tuned SQL-Llama with synthetic data to strengthen SQL error detection and repair.

2 Related Works

In this project, we build upon MAC-SQL, a multi-agent Text-to-SQL framework where specialized agents collaboratively improve SQL generation [4]. Our work focuses particularly on strengthening the

selector and decomposer components. Related approaches such as DIN-SQL [2] demonstrate that decomposing a query into smaller reasoning steps can greatly improve SQL accuracy, motivating our redesign of the decomposer prompts. Additionally, prior research in prompt engineering, including DAIL-SQL [1], shows that the structure and content of prompts strongly influence LLM performance, reinforcing our need to systematically revise both selector and decomposer prompts.

Beyond prompt design, advances in multi-agent LLM systems further inform our methodology. AutoGen [5] and a recent multi-agent Text-to-SQL framework for real-world applications [3] provide concrete evidence that cooperative LLM agents—specialized for tasks such as schema understanding, SQL generation, and verification—yield practical benefits in robustness and usability. Inspired by this paradigm, our work adopts a similar collaborative structure while emphasizing usability through interactive visualization and a transparent agent workflow.

3 Methodology

3.1 Reinforced Agents

A primary weakness of the baseline Selector is that its output is not strictly respected. Even when a subset of columns is selected, the baseline code unconditionally injects Primary Keys (PK) and Foreign Keys (FK) and pads the table to approximately 6 columns. This re-introduces irrelevant noises and dilutes schema compression.

Prompt-Level Strictness. We redesigned the Selector’s prompt template to impose rigid constraints on the agent’s reasoning. The prompt explicitly mandates that the agent:

- (1) Select only tables strictly required for JOIN operations or SELECT clauses.
- (2) Include only columns semantically related to the user’s natural language question.
- (3) Explicitly mark irrelevant tables with a `drop_all` tag.
- (4) Adhere strictly to a valid JSON output format to ensure parseability.

Table-level Strictness. The `drop_all` command results in the complete removal of the table, correcting the baseline behavior where a small prefix of columns remained visible.

Column-level Strictness. When the Selector returns a specific list of columns, the prompt includes only those columns, eliminating heuristic padding. To reduce mismatches caused by formatting differences (e.g., `first_name` vs. `FirstName`), we apply a canonical transformation that removes non-alphanumeric characters and normalizes case. Column selection is applied under this canonical equivalence to improve robustness to minor LLM naming variations.

Consistent Pruning Policy. The baseline relies on heuristic thresholds derived from schema statistics to trigger pruning, which causes inconsistent behavior across datasets. In our revised system, pruning is the default behavior: `is_need_prune()` always returns true (unless explicitly disabled). This ensures consistent schema scoping and avoids dataset-dependent branching.

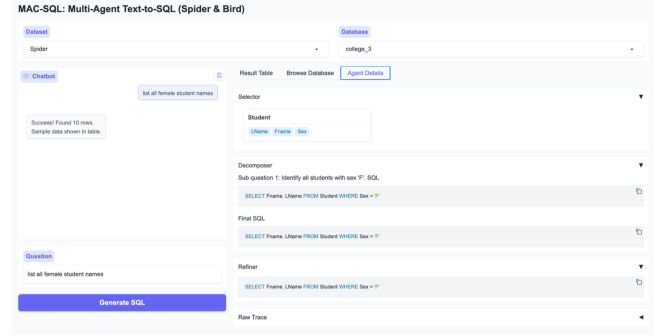


Figure 2: An example of Refined Agents. The Selector only selects the tables and columns that are necessary to the question. The Decomposer breaks questions into sub-questions and provides sub-queries.

Improved Decomposer Output Extraction. LLM-based decompositions often return structured reasoning mixed with SQL segments. The baseline blindly extracts SQL from the entire response, which fails when the model outputs intermediate query fragments. We implemented a lightweight heuristic:

- (1) we add the sub query step for each sub question and sparse these queries separately
- (2) If the response contains a “SQL” marker, we parse only the trailing region without reasoning threads. This reduces extraction failures without requiring rigid prompt templates.

3.2 Fine-tune SQL-Llama

The baseline project created an open-source alternative to GPT-4, SQL-Llama, by fine-tuning Code Llama 7B with the agent instruction dataset derived from training data based on BIRD and Spider. However, when compared to GPT-4, there remains a notable potential for enhancement. To address this, we implemented a Supervised Fine-Tuning (SFT) strategy on Llama-7B, augmenting its training with a targeted synthetic dataset designed to rectify specific error patterns observed in the Spider and BIRD benchmarks.

Targeted Error Analysis. We identified two recurring failure modes where the pre-trained Llama model consistently diverged from ground-truth SQL:

- (1) **Aggregation Logic (Spider Benchmark):** The model frequently omitted the HAVING clause in queries requiring post-aggregation filtering, confusing it with the WHERE clause.
- (2) **Date Transformation (BIRD Benchmark):** The model struggled with dialect-specific date operations, failing to correctly apply functions (e.g., `STRFTIME` or `casting`) when querying temporal data (e.g., “prior to 1998”).

Synthetic Data Generation via Template Injection. To correct these deficiencies without over-fitting to specific database schemas, we developed a Template-Based Data Augmentation pipeline. This method isolates the syntactic structure of a failed query and systematically generates variations across different schema domains.

The process functions as follows:

- **Template Extraction:** We abstract the logic of an incorrect query into a generalized template by replacing domain-specific entities with variable placeholders.
 - *Original Question (Spider):* “How many countries has more than 2 car makers?”
Abstracted Template: “How many {parent_phrase} has more than {number} {child_phrase}?”
 - *Original Question (BIRD):* “How many percent of ‘Gold’ cards were issued prior to 1998?”
Abstracted Template: “How many percent of {parent_phrase} were issued prior to {child_phrase}?”
- **Schema Instantiation:** Using a Python generation script, we programmatically map these templates to valid existing tables and columns in Spider/BIRD datasets to create several instruction-tuning pairs. We also utilize template mappings for the ground-truth SQL query labels on all the instruction-tuning pairs. This ensures the model learns the logic for HAVING clause or date comparison, rather than over-fitting and memorizing specific table names.
- **Dataset Construction:** We generated a synthetic corpus containing approximately 1,500 new instruction-tuning pairs for the Spider failure pattern and 100 new instruction-tuning pairs for the BIRD failure pattern. Each sample pairs the templated natural language question with the corrected, syntactically rigorous templated SQL query.

Implementation and Hardware Constraints. We integrated the fine-tuning process into the MAC-SQL repository, utilizing the existing training pipeline. However, we encountered significant implementation and hardware bottlenecks during execution.

- **Insufficient Training Data:** While we were able to generate around 1,500 records for fine-tuning the Spider failure pattern which is likely sufficient, we could only generate around 100 records for the BIRD failure pattern. This is an issue because fine-tuning SQL-Llama to fix the BIRD failure pattern will require more than this, considering it took approximately 3,000 to fine-tune Code Llama 7B and obtain SQL-Llama. The reason there are so few training records generated for the BIRD pattern is that it requires a column with date values for the child phrase, which there may not be enough of in BIRD. In the future, we will need more data so that this is not a bottleneck.
- **Memory Overhead:** Standard training environments (e.g., Google Colab Free Tier with ~13 GB VRAM) proved insufficient for loading the model weights and gradients, resulting in Out-Of-Memory (OOM) failures during the backward pass.
- **High-Memory Resolution:** To resolve this, we migrated the training workload to a high-memory environment (Google Colab Pro+), utilizing an A100 GPU with 40 GB of VRAM. This capacity allowed for adequate batch sizes and context window retention, ensuring the model could process the verbose SQL schema descriptions without truncation.
- **Training Times:** Even when compute requirements are satisfied with Google Colab Pro, the platform still has the issue of automatically disconnecting runtimes after a certain amount of time has passed. There is no way to stop this from happening, which means that training runs that take too

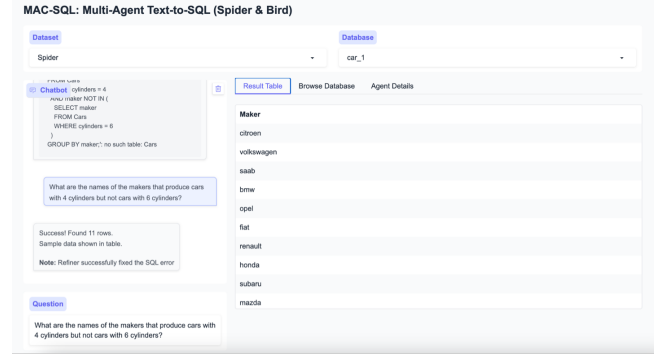


Figure 3: UI demo showing the Gradio interface.

long will be interrupted before they can finish. Training environments that allow for long-running jobs will be required in the future.

3.3 Interactive UI

We developed a modular, web-based interface using Gradio to make the multi-agent pipeline transparent and accessible. The UI exposes intermediate states, enables real-time verification, and supports schema exploration.

- **Observability:** The UI visualizes the full execution trace (Selector → Decomposer → Refiner). Runtime failures, such as SQLite syntax errors, are surfaced alongside the agents’ autonomous repair attempts.
- **Visual Reasoning:** Selector outputs are rendered as HTML cards with color-coded column badges. The Decomposer trace is presented in formatted Markdown, highlighting ordered reasoning steps.
- **Interactive Verification:** Generated SQL is executed immediately and displayed in a scrollable DataFrame. A “Browse Database” tab allows users to inspect raw data to validate assumptions about data formats.

4 Evaluation

4.1 Datasets

We conduct evaluation on two customized datasets of BIRD and Spider. The BIRD subset contains 195 questions with three difficulty levels (simple, moderate, challenging). The Spider subset contains 200 questions with four levels (easy, medium, hard, extra hard).

4.2 Metrics

Following the baseline project evaluation practice, we report two execution-focused metrics:

- **Execution Accuracy (EX):** the fraction of examples for which the predicted SQL and the gold SQL return identical execution results.
- **Valid Efficiency Score (VES):** the efficiency of *valid* predicted SQL, computed only over predictions whose result sets match the gold results.

Table 1: BIRD performance by difficulty (Dev, $N=195$).

Difficulty	Count	EX (%)	VES (%)
Simple	117	67.50	71.80
Moderate	67	60.20	66.00
Challenging	11	39.10	46.68
Total	195	63.39	68.39

Table 2: Spider execution accuracy by difficulty (Dev, $N=200$).

Difficulty	Count	EX (%)
Easy	81	94.50
Medium	44	93.20
Hard	57	90.90
Extra hard	18	84.08
Total	200	92.25

Table 3: BIRD error distribution ($N=71$ errors). Counts are computed from percentages with rounding.

Error Type	Count	Percentage
Database Misunderstand	56	79.0%
Question Misunderstand	11	16.0%
Semantic Correct	2	3.0%
Other	2	2.0%

We do not report Exact Match (EM) because it penalizes syntactically different but semantically equivalent SQL queries.

4.3 Results

Table 1 summarizes performance on BIRD subset. Overall, the system achieves 63.39% EX and 68.39% VES. As expected, performance declines as difficulty increases: 67.50% on simple questions, 60.20% on moderate questions, and 39.10% on challenging questions.

Table 2 summarizes results on Spider subsets. The system achieves 92.25% overall execution accuracy, with consistently high accuracy across difficulty bands. Even in the extra-hard subset, accuracy remains at 84.08%.

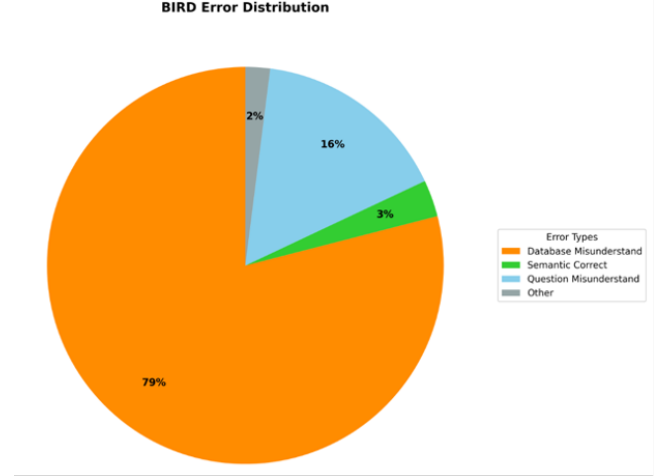
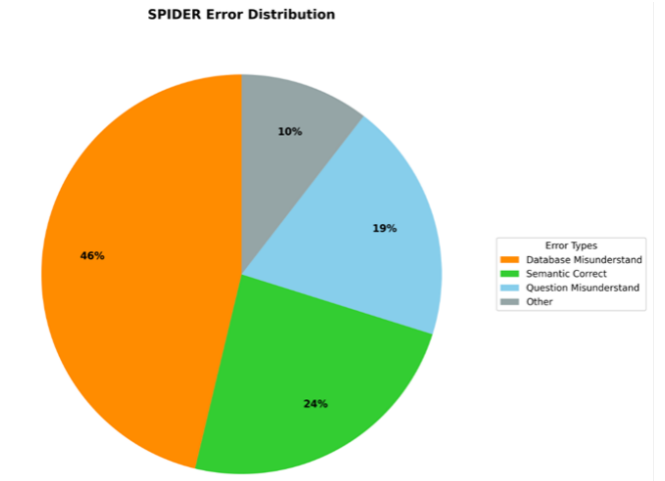
4.4 Error Analysis

To better understand failure modes, We classified errors into eight categories: Gold Error (incorrect ground truth), Database Misunderstand (schema/value misinterpretation), Semantic Correct (valid alternative SQL), Question Misunderstand (logical interpretation errors), Evidence Misunderstand (external knowledge issues), Dirty Database Values (data quality), Schema Linking Error (incorrect schema mapping), and Other (uncategorizable errors).

On BIRD, we observe 71 errors out of 195 examples (36.41% error rate). As shown in Table 3, Database Misunderstand is the dominant failure mode (79.0%), suggesting that real-world schema complexity and value ambiguity remain the primary bottlenecks.

Table 4: Spider error distribution ($N=16$ errors). Counts are computed from percentages with rounding.

Error Type	Count	Percentage
Database Misunderstand	7	46.3%
Semantic Correct	4	23.9%
Question Misunderstand	3	19.4%
Other	2	10.4%

**Figure 4: Distribution of error types in Bird****Figure 5: Distribution of error types in Spider.**

Spider contains 16 errors out of 200 examples (8.0% error rate). Table 4 shows a more balanced error profile, with fewer schema-related failures and a higher share of Semantic Correct cases, indicating that many “errors” may reflect alternative but valid SQL formulations.

4.5 Discussion

Our evaluation highlights three main findings:

- **Dataset complexity strongly impacts accuracy.** The gap between Spider (92.25%) and BIRD (63.39%) indicates that BIRD’s real-world schemas and ambiguity substantially increase difficulty.
- **Schema/value interpretation is the dominant bottleneck on BIRD.** Database Misunderstand accounts for 79.0% of BIRD errors, suggesting that improving schema grounding and value linking is critical for complex databases.
- **Hard queries remain challenging.** Performance drops sharply on BIRD challenging questions (39.10%), implying that multi-step reasoning and decomposition strategies require further strengthening.

5 Conclusion

In this paper, we present MAC-SQL Pro, a reinforced multi-agent Text-to-SQL framework. By implementing strict schema scoping with canonical matching, we eliminated the noise introduced by the baseline’s heuristic padding. We further optimized performance by fine-tuning a SQL-Llama model on a targeted synthetic dataset to address specific aggregation and date-logic failures. Finally, we bridged the gap between research and usability by developing a transparent, interactive UI. Our evaluation demonstrates that these engineering and algorithmic refinements yield a more robust and explainable Text-to-SQL system.

References

- [1] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yanjian Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. arXiv:2308.15363
- [2] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. arXiv:2304.11015 [cs.CL] doi:10.48550/arXiv.2304.11015 To appear in NeurIPS 2023.
- [3] Chen Shen, Jin Wang, Sajjadur Rahman, and Eser Kandogan. 2024. Demonstration of Multi-Agent Framework for Text-to-SQL Applications with Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*. ACM, New York, NY, USA, 5280–5283. doi:10.1145/3627673.3679216
- [4] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, LinZheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 540–557. <https://aclanthology.org/2025.coling-main.36/>
- [5] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the First Conference on Language Modeling (COLM 2024)*. doi:10.48550/arXiv.2308.08155

A Appendix: Prompts and Key Implementation Snippets

This appendix provides the core prompts and representative implementation excerpts used to reinforce the Selector and Decomposer components in MAC-SQL Pro.

A.1 Selector Prompt Template

```
selector_template = """
As an experienced and professional database administrator,
your task is to analyze a user question and a database schema
to provide relevant information. The database schema consists
of table descriptions, each containing multiple column
descriptions.
Your goal is to identify the relevant tables and columns
based on the user question and evidence provided.

[Instruction]:
1. ANALYZE the question and evidence carefully.
2. SELECT only the tables required for the JOIN or SELECT.
3. SELECT only the columns related to the question.
4. EXCLUDE all other unrelated columns.
5. If a table is not needed, mark it as "drop_all".
6. The output must be valid JSON.

Requirements:
1. Only select the tables and columns strictly necessary to
answer the question.
2. Do not include extra tables and columns just because the
table is small.
3. If a table is completely irrelevant, mark it as "drop_all".
"""
```

A.2 Selector Agent (Excerpt)

The excerpt below illustrates how the Selector (i) loads schema metadata and value examples, (ii) invokes the LLM using the selector prompt, and (iii) enforces strict schema pruning behavior (including drop_all support and no PK/FK padding). Check our project link for detailed code.

```
class Selector(BaseAgent):
    name = SELECTOR_NAME
    description = "Get database description and if need,
    extract relative tables & columns"

    def __init__(
        self,
        data_path: str,
        tables_json_path: str,
        model_name: str,
        dataset_name: str,
        lazy: bool = False,
        without_selector: bool = False
    ):
        super().__init__()
        self.data_path = data_path.strip("/").strip("\\")
        self.tables_json_path = tables_json_path
        self.model_name = model_name
        self.dataset_name = dataset_name
        self.db2infos = {}
        self.db2dbjsons = {}
        self.init_db2jsons()
        if not lazy:
            self._load_all_db_info()
        self._message = {}
        self.without_selector = without_selector

    def init_db2jsons(self):
        if not os.path.exists(self.tables_json_path):
            raise FileNotFoundError(f"tables.json not found in
            {self.tables_json_path}")
        data = load_json_file(self.tables_json_path)
        for item in data:
            db_id = item["db_id"]
```

```
        table_names = item["table_names"]
        item["table_count"] = len(table_names)
        column_count_lst = [0] * len(table_names)
        for tb_idx, col in item["column_names"]:
            if tb_idx >= 0:
                column_count_lst[tb_idx] += 1
        item["max_column_count"] = max(column_count_lst) if
        column_count_lst else 0
        item["total_column_count"] = sum(column_count_lst)
        item["avg_column_count"] = (sum(column_count_lst) //
        len(table_names)) if table_names else 0
        self.db2dbjsons[db_id] = item

    def _get_column_attributes(self, cursor, table):
        cursor.execute(f"PRAGMA table_info(`{table}`)")
        columns = cursor.fetchall()
        column_names = []
        column_types = []
        for column in columns:
            column_names.append(column[1])
            column_types.append(column[2])
        return column_names, column_types

    def _get_value_examples_str(self, values: List[object],
        col_type: str):
        if not values:
            return ""
        if len(values) > 10 and col_type in ["INTEGER", "REAL",
        "NUMERIC", "FLOAT", "INT"]:
            return ""
        vals = []
        has_null = False
        for v in values:
            if v is None:
                has_null = True
            else:
                tmp_v = str(v).strip()
                if tmp_v != "":
                    vals.append(v)
        if not vals:
            return ""
        if col_type in ["TEXT", "VARCHAR"]:
            new_values = []
            for v in vals:
                if not isinstance(v, str):
                    new_values.append(v)
            else:
                if self.dataset_name == "spider":
                    v = v.strip()
                    if v == "" or "https://" in v or "http://" in
                    v or is_email(v):
                        continue
                else:
                    new_values.append(v)
            vals = new_values
            tmp_vals = [len(str(a)) for a in vals]
            if not tmp_vals or max(tmp_vals) > 50:
                return ""
        if not vals:
            return ""
        vals = vals[:6]
        if is_valid_date_column(vals):
            vals = vals[:1]
        if has_null:
            vals.insert(0, None)
        return str(vals)

    def _prune(self, db_id: str, query: str, db_schema: str,
        db_fk: str, evidence: str = None) -> dict:
        prompt = selector_template.format(
            db_id=db_id, query=query, evidence=evidence,
            desc_str=db_schema, fk_str=db_fk
        )
        word_info = extract_world_info(self._message)
        reply = LLM_API_FUC(prompt, **word_info)

        print(f"\n[Selector] Raw LLM Reply (first 1200
        chars):\n{str(reply)[:1200]}\n")
```

```

def clean_json_text(text: str) -> str:
    text = (text or "").strip()
    text = re.sub(r"^\s*````(?:json)?\s*", "", text,
        flags=re.IGNORECASE)
    text = re.sub(r"\s*````\s*", "", text)
    return text.strip()

clean_reply = clean_json_text(str(reply))

try:
    parsed = parse_json(clean_reply)
    if isinstance(parsed, dict) and parsed:
        return parsed
except Exception:
    pass

try:
    parsed = json.loads(clean_reply)
    if isinstance(parsed, dict) and parsed:
        return parsed
except Exception:
    pass

print("[Selector] JSON parsing FAILED. Returning {}.")
return {}

```

A.3 Decomposer Prompt Template

decompose_template_bird = ""
 Given a Database schema description, a knowledge Evidence and the
 Question, you need to use
 valid SQLite and understand the database and knowledge.

Your task is to:

1. Decompose the complex question into sub-questions.
2. Generate a valid SQL query for EACH sub-question.
3. Combine them into a "Final SQL" at the end.

IMPORTANT:

1. Answer STRICTLY in the format below.
2. Do NOT output any introductory text, reasoning paragraphs, or "Here is the result".
3. Just output the Sub questions and SQL.

Follow this format strictly:

Sub question 1: <reasoning>
 <SQL>
 Sub question 2: <reasoning>
 <SQL>

Final SQL

<SQL>

Constraints

- In `SELECT <column>`, just select needed columns in the Question without any unnecessary column or value
- In `FROM <table>` or `JOIN <table>`, do not include unnecessary table
- If use max or min func, `JOIN <table>` FIRST, THEN use `SELECT MAX(<column>)` or `SELECT MIN(<column>)`
- If [Value examples] of <column> has 'None' or None, use `JOIN <table>` or `WHERE <column> is NOT NULL` is better
- If use `ORDER BY <column> ASC|DESC`, add `GROUP BY <column>` before to select distinct values

question: "What are the names of the makers that produce cars with 4 cylinders but not cars with 6 cylinders?"

As shown in Figure 6, the Decomposer initially generates a SQL query that joins cars_data and model_list using model_list.MakeId. SQLite reports an execution failure because model_list.MakeId does not exist in the database schema. The Refiner then repairs the query by (i) adjusting the join path to match the schema and (ii) rewriting joins using the correct key columns across the relevant tables (car_names, model_list, and car_makers). After correction, the refined SQL executes successfully and returns a valid maker list in the UI.

The interface shows the following components:

- Database:** Spider, car_1
- Result Table:** A table with columns 'Maker' and 'Cylinders'. It lists makers: citroen, volkswagen, saab, bmw, opel, fiat, renault, honda, subaru, mazda.
- Question:** "What are the names of the makers that produce cars with 4 cylinders but not cars with 6 cylinders?"
- Original SQL (from Decomposer):**

```

SELECT DISTINCT `Maker` FROM cars_data
JOIN model_list ON cars_data.Id = model_list.MakeId
WHERE cars_data.Cylinders = 4
AND `Maker` NOT IN (
    SELECT DISTINCT `Maker` FROM cars_data
    JOIN model_list ON cars_data.Id = model_list.MakeId
    WHERE cars_data.Cylinders = 6
)

```
- Error detected:**

```

SQL execution error: Execution failed on sql 'SELECT DISTINCT `Maker` FROM cars_data JOIN model_list ON cars_data.Id = model_list.MakeId WHERE cars_data.Cylinders = 4 AND `Maker` NOT IN ( SELECT DISTINCT `Maker` FROM cars_data JOIN model_list ON cars_data.Id = model_list.MakeId WHERE cars_data.Cylinders = 6 ); no such column: model_list.MakeId

```
- Refined SQL:**

```

-- Corrected SQL to find makers that produce cars with 4 cylinders but not with 6 cylinders
SELECT DISTINCT cm.Maker
FROM cars_data cd
JOIN car_names cn ON cd.Id = cn.MakeId
JOIN model_list ml ON cn.Model = ml.Model
JOIN car_makers cm ON ml.Maker = cm.Id
WHERE cd.Cylinders = 4
AND cm.Maker NOT IN (
    SELECT DISTINCT cm2.Maker
    FROM cars_data cd2
    JOIN car_names cn2 ON cd2.Id = cn2.MakeId
    JOIN model_list ml2 ON cn2.Model = ml2.Model
    JOIN car_makers cm2 ON ml2.Maker = cm2.Id
    WHERE cd2.Cylinders = 6
)

```

Figure 6: Refiner correction example on Spider (car_1). The Decomposer produces SQL that fails execution due to an invalid join key, and the Refiner rewrites the query using schema-consistent joins, producing executable SQL and the expected result table.

B Appendix: Refiner Repair Example (Decomposer-to-Executable SQL)

This appendix provides a concrete example showing how the Refiner detects and repairs execution-time failures from the Decomposer. We use a Spider example from database car_1 with the

C Appendix: BIRD Error Examples

C.1 Example 1: Question Misunderstand

DB ID	codebase_community
Difficulty	simple
Error Type	Question Misunderstand

Question. Which user has a higher reputation, Harlan or Jarrod Dixon?

Evidence. “Harlan” and “Jarrod Dixon” are both DisplayName; highest reputation refers to MAX(Reputation).

Gold SQL.

```
SELECT DisplayName
FROM users
WHERE DisplayName IN ('Harlan', 'Jarrod Dixon')
AND Reputation = (
  SELECT MAX(Reputation)
  FROM users
  WHERE DisplayName IN ('Harlan', 'Jarrod Dixon')
)
```

Predicted SQL.

```
SELECT `DisplayName`, `Reputation`
FROM users
WHERE `DisplayName` = "Harlan"
```

Error Description. The prediction fails to compare both candidates and does not apply MAX(Reputation) across the two names (it only filters for “Harlan”), so it cannot answer “which has higher reputation.”

C.2 Example 2: Database Misunderstand

DB ID	codebase_community
Difficulty	simple
Error Type	Database Misunderstand

Question. What is the total score of the posts edited by Yevgeny and include the user’s website URL.

Evidence. “Yevgeny” is the DisplayName; edited refers to LastEditorUserId.

Gold SQL.

```
SELECT SUM(T1.Score), T2.WebsiteUrl
FROM posts AS T1
INNER JOIN users AS T2 ON T1.OwnerUserId = T2.Id
WHERE T2.DisplayName = 'Yevgeny'
GROUP BY T2.WebsiteUrl
```

Predicted SQL.

```
SELECT SUM(T1.`Score`)
FROM posts AS T1
INNER JOIN users AS T2 ON T1.`LastEditorUserId` = T2.`Id`
WHERE T2.`DisplayName` = 'Yevgeny'
```

Error Description. The prediction fails to compare both candidates and does not apply MAX(Reputation) across the two names (it only filters for “Harlan”), so it cannot answer “which has higher reputation.”

DB ID	car_1
Difficulty	extra hard
Exec Result	0
Error Type	Question Misunderstand

D Spider Error Examples

D.1 Example 1: Question Misunderstand

Question. What is the maximum horsepower and the make of the car models with 3 cylinders?

Gold SQL.

```
SELECT T2.horsepower, T1.Make
FROM CAR_NAMES AS T1
JOIN CARS_DATA AS T2 ON T1.MakeId = T2.Id
WHERE T2.cylinders = 3
ORDER BY T2.horsepower DESC
LIMIT 1;
```

Predicted SQL.

```
SELECT T2.`Make`
FROM cars_data AS T1
INNER JOIN car_names AS T2 ON T1.`Id` = T2.`MakeId`
WHERE T1.`Cylinders` = 3
AND T1.`Horsepower` = (
  SELECT MAX(`Horsepower`)
  FROM cars_data
  WHERE `Cylinders` = 3
)
```

Error Description. The prediction under-selects required outputs: it returns only the make and omits the maximum horsepower value requested in the question (missing one output column).

D.2 Example 2: Database Misunderstand

DB ID	world_1
Difficulty	easy
Exec Result	0
Error Type	Database Misunderstand

Question. Give the country codes for countries in which people speak languages that are not English.

Gold SQL.

```
SELECT DISTINCT CountryCode
FROM countrylanguage
WHERE LANGUAGE != "English"
```

Predicted SQL.

```
SELECT `CountryCode`
FROM countrylanguage
WHERE `Language` != 'English'
```

Error Description. The prediction does not apply DISTINCT, so it can return duplicate country codes when multiple non-English languages exist for the same country (missing deduplication).